Contents

Introduction 2
Installation 2
Overview 4
Project Structure 4
Platform 4
Controllers 6
Routing 6
Request object 8
Resources 10
Route wildcards 10
Status code 11
Response headers 12
Redirection 12
Route parameters 13
Getting up and running 14
Providers 15
Services 15
Dependency injection 16
Scopes 16
Combining Controller with Service 16
Modules 18
Feature modules 19
Shared modules 20
Global modules 21
Dynamic modules 22
CRUD Generator 23
Generating a new resource 23
Creating RestAPI using NestJS 24

DRAFT

Introduction

Nest (NestJS) is a framework for building efficient, scalable Node.js server-side applications. It uses progressive JavaScript, is built with and fully supports TypeScript (yet still enables developers to code in pure JavaScript) and combines elements of OOP (Object Oriented Programming), FP (Functional Programming), and FRP (Functional Reactive Programming).

Under the hood, Nest makes use of robust HTTP Server frameworks like Express (the default) and optionally can be configured to use Fastify as well!

Nest provides a level of abstraction above these common Node.js frameworks (Express/Fastify), but also exposes their APIs directly to the developer. This gives developers the freedom to use lots of third-party modules which are available for the underlying platform.

In recent years, thanks to Node.js, JavaScript has become the "**lingua franca**" of the web for both front and backend applications. This has given rise to awesome projects like Angular, React and Vue, which improve developer productivity and enable the creation of fast, testable, and extensible frontend applications. However, while plenty of superb libraries, helpers, and tools exist for Node (and server-side JavaScript), none of them effectively solve the main problem of architecture.

Nest provides an out-of-the-box application architecture which allows developers and teams to create highly testable, scalable, loosely coupled, and easily maintainable applications. The architecture is heavily inspired by Angular.

Installation

To get started, you can either scaffold the project with the Nest CLI, or clone a starter project (both will produce the same outcome).

To scaffold the project with the Nest CLI, run the following commands. This will create a new project directory, and populate the directory with the initial core Nest files and supporting modules, creating a conventional base structure for your project. Creating a new project with the Nest CLI is recommended for first-time users. We'll continue with this approach in First Steps.

```
npm i -g @nestjs/cli
nest new project-name
```

Note: To create a new TypeScript project with stricter feature set, pass the **--strict** flag to the nest new command.

Alternatively, to install the TypeScript starter project with Git:

DRAFT

```
git clone https://github.com/nestjs/typescript-starter.git project
cd project
npm install
```

After installation we can boot the server using following command,

```
npm run start
```

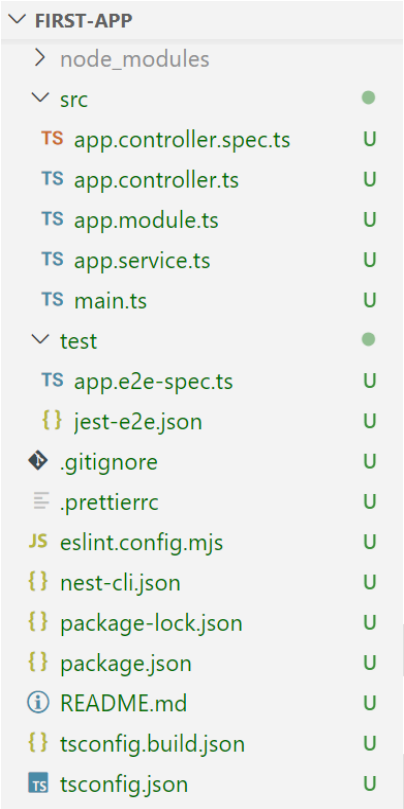
After this open the browser and navigate to http://localhost:3000

DRAFT

Overview

In this chapter, you'll learn the core fundamentals of Nest. To get familiar with the essential building blocks of Nest applications, we'll build a basic CRUD application with features that cover a lot of ground at an introductory level.

Project Structure



Let's start by exploring the structure of NestJS project,

Folders:

- **node_modules:** contains dependencies
- **src:** contains source code of the project
- **test:** contains files to specify tests
- **dist** (if generated): contains compiled files

Files inside project directory:

- .gitignore: specify files/folders needs to be ignored by git
- .prettierrc: configuration file for prettier.io
- eslint.config.mjs: configuration file for eslint
- nest-cli.json: config file for nest command line interface
- package.json: config file for Node Package Manager
- tsconfig.json: configuration file for TypeScript

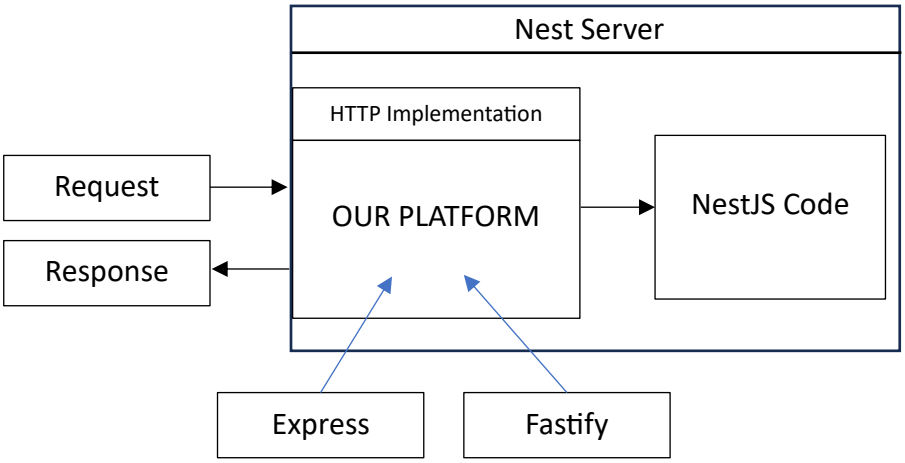
Files inside src directory:

- app.controller.spec.ts: The unit tests for the controller.
- app.controller.ts: A basic controller with a single route.
- app.module.ts: The root module of the application.
- app.service.ts: A basic service with a single method.
- main.ts: The entry file of the application which uses the core function NestFactory to create a Nest application instance.

Platform

Nest aims to be a platform-agnostic framework. Platform independence makes it possible to create reusable logical parts that developers can take advantage of across several different types of applications.

Technically, Nest is able to work with any Node HTTP framework once an adapter is created. There are two HTTP platforms supported out-of-the-box: express and fastify.



As per the figure above we can choose the one from **express** or **fastify** that best suit your needs.

We can change the platform from the package.json file, you just need to change the dependency.

DRAFT

Controllers

Controllers are responsible for handling incoming requests and sending responses back to the client.

A controller's purpose is to handle specific requests for the application. The routing mechanism determines which controller will handle each request. Often, a controller has multiple routes, and each route can perform a different action.

To create a basic controller, we use classes and decorators. Decorators link classes with the necessary metadata, allowing Nest to create a routing map that connects requests to their corresponding controllers.

Routing

In the following example, we'll use the @Controller() decorator, which is required to define a basic controller. We'll specify an optional route path prefix of **students**. Using a path prefix in the @Controller() decorator helps us group related routes together and reduces repetitive code.

For example, if we want to group routes that manage interactions with a student entity under the /students path, we can specify the students path prefix in the @Controller() decorator. This way, we don't need to repeat that portion of the path for each route in the file.

We can generate student controller using Nest CLI by executing following command,

```
nest g controller students
```

This command will create two files inside src/students folder and update src/app.module.ts file.

Above command will create a file named students.controller.ts file inside students folder in src folder, default content for the file is as follows,

src/students/students.controller.ts

```
import { Controller } from '@nestjs/common';

@Controller('students')
export class StudentsController {

}
```

To create a basic route in the controller we need to import **Get** decorator from @nestjs/common and writing decorator inside StudentsController class, here is a basic usage of Get decorator.

src/students/students.controller.ts

```
import { Controller, Get } from '@nestjs/common';

@Controller('students')
export class StudentsController {
    @Get()
    findAll(){
        return 'This action will return all students'
    }
}
```

The @Get() HTTP request method decorator placed before the findAll() method tells Nest to create a handler for a specific endpoint for HTTP requests. This endpoint is defined by the HTTP request method (GET in this case) and the route path.

The route path for a handler is determined by combining the (optional) prefix declared for the controller with any path specified in the method's decorator. Since we've set a prefix (students) for every route and haven't added any specific path in the method decorator, Nest will map GET /students requests to this handler.

As mentioned, the route path includes both the optional controller path prefix and any path string specified in the method's decorator. For example, if the controller prefix is students and the method decorator is @Get('engineering'), the resulting route will be GET /students/engineering.

In our example above, when a GET request is made to this endpoint, Nest routes the request to the user-defined findAll() method. Note that the method name we choose here is entirely arbitrary. While we must declare a method to bind the route to, Nest doesn't attach any specific significance to the method name.

We first need to introduce the concept that Nest uses two different options for manipulating responses:

- Standard (recommended):
  Using this built-in method, when a request handler returns a JavaScript object or array, it will automatically be serialized to JSON. When it returns a JavaScript primitive type (e.g., string, number, boolean), however, Nest will send just the value without attempting to serialize it. This makes response handling simple: just return the value, and Nest takes care of the rest.
  Furthermore, the response's status code is always 200 by default, except for POST requests which use 201. We can easily change this behaviour by adding the @HttpCode(...) decorator at a handler-level.
- Library Specific:

We can use the library-specific (e.g., Express) response object, which can be injected using the @Res() decorator in the method handler signature (e.g., findAll(@Res() response)).

With this approach, you have the ability to use the native response handling methods exposed by that object. For example, with Express, you can construct responses using code like response.send("Hello").status(200)

src/students/students.controller.ts (not recommended)

```
import { Controller, Get, Req, Res } from '@nestjs/common';
import { Request, Response } from 'express';

@Controller('students')
export class StudentsController {
    @Get()
    findAll(@Req() req: Request, @Res() res: Response) {
      res.send("Hello world").status(200)
    }
}
```

Nest detects when the handler is using either @Res() or @Next(), indicating you have chosen the library-specific option.

If both approaches are used at the same time, the Standard approach is automatically disabled for this single route and will no longer work as expected.

To use both approaches at the same time (for example, by injecting the response object to only set cookies/headers but still leave the rest to the framework), you must set the passthrough option to true in the @Res({ passthrough: true }) decorator.

Request object

Handlers often need access to the client's request details. Nest provides access to the request object from the underlying platform (Express by default).

You can access the request object by instructing Nest to inject it using the @Req() decorator in the handler's signature.

src/students/students.controller.ts

```
import { Controller, Get, Req } from '@nestjs/common';
import { Request } from 'express';

@Controller('students')
export class StudentsController {
    @Get()
    findAll(@Req() req: Request): string {
        console.log(req);
        return "Hello world with req object"
    }
}
```

The request object represents the HTTP request and contains properties for the query string, parameters, HTTP headers, and body (read more here).

In most cases, you don't need to manually access these properties. Instead, you can use dedicated decorators like @Body() or @Query(), which are available out of the box. Below is a list of the provided decorators and the corresponding platform-specific objects they represent.

Decorator	Platform specific object
`@Request(), @Req()`	`req`
`@Response(), @Res()`	`res`
`@Next()`	`next`
`@Session()`	`req.session`
`@Param(key?: string)`	`req.params` / `req.params[key]`
`@Body(key?: string)`	`req.body` / `req.body[key]`
`@Query(key?: string)`	`req.query` / `req.query[key]`
`@Headers(name?: string)`	`req.headers` / `req.headers[name]`
`@Ip()`	`req.ip`
`@HostParam()`	`req.hosts`

For compatibility with typings across underlying HTTP platforms (e.g., Express and Fastify), Nest provides @Res() and @Response() decorators. @Res() is simply an alias for @Response(). Both directly expose the underlying native platform response object interface.

When using them, you should also import the typings for the underlying library (e.g., @types/express) to take full advantage. Note that when you inject either @Res() or @Response() in a method handler, you put Nest into Library-specific mode for that handler, and you become responsible for managing the response.

When doing so, you must issue some kind of response by making a call on the response object (e.g., res.json(...) or res.send(...)), or the HTTP server will hang.

DRAFT

Resources

Earlier, we defined an endpoint to fetch the students resource (GET route). We'll typically also want to provide an endpoint that creates new records. For this, let's create the POST handler:

src/students/students.controller.ts

```
import { All, Controller, Get, Post } from '@nestjs/common';

@Controller('students')
export class StudentsController {
    @Get()
    findAll(): string {
        return "Hello world with get method"
    }

    @Post()
    insert(): string {
        return "Hello world with post method"
    }

    @All()
    anymethod(): string{
        return "Hello world with any method"
    }
}
```

It's that simple. Nest provides decorators for all of the standard HTTP methods: @Get(), @Post(), @Put(), @Delete(), @Patch(), @Options(), and @Head(). In addition, @All() defines an endpoint that handles all of them.

Route wildcards

Pattern-based routes are also supported in NestJS. For example, the asterisk (*) can be used as a wildcard to match any combination of characters in a route at the end of a path. In the following example, the findAll() method will be executed for any route that starts with abcd/, regardless of the number of characters that follow.

src/students/students.controller.ts

```
import { All, Controller, Get, Post } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Get('abcd/*')
    findAll() {
        return 'This route uses a wildcard';
    }

}
```

The 'abcd/*' route path will match abcd/, abcd/123, abcd/abc, abcd/xyz/123 and so on.

The hyphen (-) and the dot (.) are interpreted literally by string-based paths.

This approach works on both Express and Fastify. However, with the latest release of Express (v5), the routing system has become more strict. In pure Express, you must use a named wildcard to make the route work—for example, abcd/*splat, where splat is simply the name of the wildcard parameter and has no special meaning. You can name it anything you like. That said, since Nest provides a compatibility layer for Express, you can still use the asterisk (*) as a wildcard.

When it comes to asterisks used in the middle of a route, Express requires named wildcards (e.g., ab{*splat}cd), while Fastify does not support them at all.

Status code

As mentioned earlier, the default status code for responses is always 200, except for POST requests, which default to 201. You can easily change this behaviour by using the @HttpCode(...) decorator at the handler level.

Often, your status code isn't static but depends on various factors. In that case, you can use a library-specific response (inject using @Res()) object (or, in case of an error, throw an exception).

DRAFT

src/students/students.controller.ts

```
import { Controller, Get, HttpCode } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Get()
    @HttpCode(204)
    findAll() {
        return 'This will be returned with 204 status code';
    }

}
```

Response headers

To specify a custom response header, you can either use a @Header() decorator or a library-specific response object (and call res.header() directly).

src/students/students.controller.ts

```
import { Controller, Header, Post } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Post()
    @Header('Cache-Control', 'no-store')
    findAll() {
        return 'Hello world';
    }

}
```

Redirection

To redirect a response to a specific URL, you can either use a @Redirect() decorator or a library-specific response object (and call res.redirect() directly).

@Redirect() takes two arguments, url and statusCode, both are optional. The default value of statusCode is 302 (Found) if omitted.

src/students/students.controller.ts

```
import { Controller, Get, Redirect } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Get()
    @Redirect('http://darshan.ac.in',302)
    test(){}
}
```

Route parameters

Routes with static paths won't work when you need to accept dynamic data as part of the request (e.g., GET /students /1 to get the student with id 1). To define routes with parameters, you can add route parameter tokens in the route path to capture the dynamic values from the URL. The route parameter token in the @Get() decorator example below illustrates this approach. These route parameters can then be accessed using the @Param() decorator, which should be added to the method signature.

src/students/students.controller.ts

```
import { Controller, Get, Param } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Get(':id')
    findOne(@Param() params: any): string {
        console.log(params.id);
        return `This action returns a student with id = ${params.id}`;
    }
}
```

The @Param() decorator is used to decorate a method parameter (in the example above, params), making the route parameters accessible as properties of that decorated method parameter inside the method. As shown in the code, you can access the id parameter by referencing params.id. Alternatively, you can pass a specific parameter token to the decorator and directly reference the route parameter by name within the method body.

src/students/students.controller.ts

```
import { Controller, Get, Param } from '@nestjs/common';

@Controller('students')
export class StudentsController {

    @Get(':id')
    findOne(@Param('id') id: string): string {
        return `This action returns a student with id = ${id}`;
    }
}
```

Getting up and running

If you create a controller manually (means **without** the nest g controller command), you need to perform certain steps in order to make it up and running.

Even with the StudentsController fully defined, Nest doesn't yet know about it and won't automatically create an instance of the class.

Controllers must always be part of a module, which is why we include the controllers array within the @Module() decorator. Since we haven't defined any other modules apart from the root AppModule, we'll use it to register the StudentsController:

src/app.module.ts

```
import { Module } from '@nestjs/common';
import { StudentsController } from './students/students.controller';

@Module({
  controllers: [StudentsController]
})
export class AppModule {}
```

Providers

Providers are a core concept in Nest. Many of the basic Nest classes, such as services, repositories, factories, and helpers, can be treated as providers.

The key idea behind a provider is that it can be injected as a dependency, allowing objects to form various relationships with each other.

The responsibility of "wiring up" these objects is largely handled by the Nest runtime system.

In the previous chapter, we created a simple StudentsController. Controllers should handle HTTP requests and delegate more complex tasks to providers.

Providers are plain JavaScript classes declared as providers in a NestJS module.

Services

Let's begin by creating a simple StudentsService. This service will handle data storage and retrieval, and it will be used by the StudentsController. Because of its role in managing the application's logic, it's an ideal candidate to be defined as a provider.

We can create services manually by creating a file named students.service.ts but creating service manually is not recommended, instead we can create a service file with NestCLI using following command,

```
nest g service students
```

Above command will create two files named students.service.ts and students.service.spec.ts in the src/students folder, it will also add StudentsService as a provider in src/app.module.ts file.

src/app.module.ts

```
. . .
import { StudentsService } from './students/students.service';

@Module({
  . . .
  providers: [. . . , StudentsService],
})
export class AppModule {}
```

src/students/students.service.ts

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class StudentsService {}
```

Dependency injection

Nest is built around the powerful design pattern known as Dependency Injection. (similar to Angular's Dependecy Injection)

In Nest, thanks to TypeScript's capabilities, managing dependencies is straightforward because they are resolved based on their type. In the example below, Nest will resolve the studentsService by creating and returning an instance of StudentsService (or, in the case of a singleton, returning the existing instance if it has already been requested elsewhere). This dependency is then injected into your controller's constructor (or assigned to the specified property):

src/students/students.controller.ts

```
import { Controller } from '@nestjs/common';
import { StudentsService } from './students.service';

@Controller('students')
export class StudentsController {

    constructor(private readonly studentsService: StudentsService) {}

}
```

Scopes

Providers typically have a lifetime ("scope") that aligns with the application lifecycle. When the application is bootstrapped, each dependency must be resolved, meaning every provider gets instantiated. Similarly, when the application shuts down, all providers are destroyed. However, it's also possible to make a provider request-scoped, meaning its lifetime is tied to a specific request rather than the application's lifecycle.

Combining Controller with Service

We can write our methods/logic in the service file and call this from the controller, lets try adding findAll and findOne method to our service and call the methods from the controller.

src/students/students.service.ts

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class StudentsService {

  findAll() {
    return `This action returns all students`;
  }

  findOne(id: number) {
    return `This action returns a #${id} student`;
  }

}
```

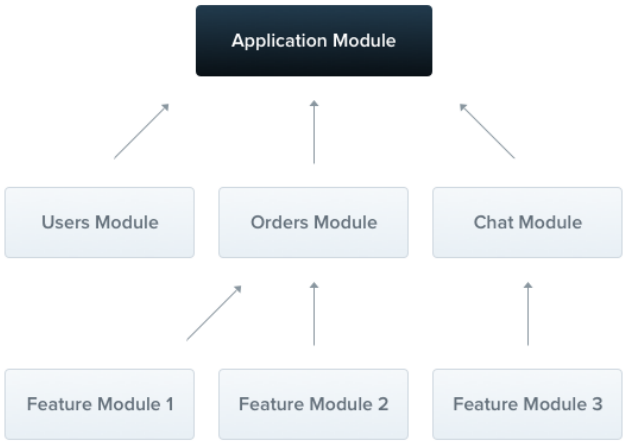
We can then inject this service to our controller and call the methods of service in our controller.

src/students/students.controller.ts

```
import { Controller, Get, Param } from '@nestjs/common';
import { StudentsService } from './students.service';

@Controller('students')
export class StudentsController {

    constructor(private readonly studentsService: StudentsService) {}

    @Get()
    findAll(){
        return this.studentsService.findAll();
    }

    @Get(':id')
    findOne(@Param('id') id: number): string {
        return this.studentsService.findOne(id);
    }
}
```

Modules

A module is a class that is annotated with the @Module() decorator. This decorator provides metadata that Nest uses to organize and manage the application structure efficiently.



Every Nest application has at least one module, the root module, which serves as the starting point for Nest to build the application graph. This graph is an internal structure that Nest uses to resolve relationships and dependencies between modules and providers. While small applications might only have a root module, this is generally not the case. Modules are highly recommended as an effective way to organize your components. For most applications, you'll likely have multiple modules, each encapsulating a closely related set of capabilities.

The @Module() decorator takes a single object with properties that describe the module:

`providers`	the providers that will be instantiated by the Nest injector and that may be shared at least across this module
`controllers`	the set of controllers defined in this module which have to be instantiated
`imports`	the list of imported modules that export the providers which are required in this module
`exports`	the subset of `providers` that are provided by this module and should be available in other modules which import this module. You can use either the provider itself or just its token (`provide` value)

The module **encapsulates** providers by default, meaning you can only inject providers that are either part of the current module or explicitly exported from other imported modules. The exported providers from a module essentially serve as the module's public interface or API.

Feature modules

In our example, the StudentsController and StudentsService are closely related and serve the same application domain. It makes sense to group them into a feature module.

A feature module organizes code that is relevant to a specific feature, helping to maintain clear boundaries and better organization. This is particularly important as the application or team grows, and it aligns with the SOLID principles.

Next, we'll create the StudentsModule to demonstrate how to group the controller and service.

We can create module manually by creating a students.module.ts file with @Module decorator, but creating module manually is not recommended, we can also create module using NestCLI by using following command,

```
nest g module students
```

This command will create a students.module.ts file with boiler-plate code in src/students folder and update the src/module.ts file.

```
src/students/students.module.ts
```

```
import { Module } from '@nestjs/common';

@Module({})
export class StudentsModule {}
```

```
src/app.module.ts
```

```
...
import { StudentsModule } from './students/students.module';

@Module({
  imports: [. . . , StudentsModule],
  . . .
})
export class AppModule {}
```

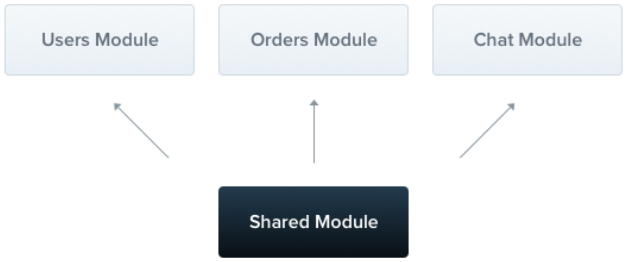
Now we need to import StudentsService and StudentsController in students.module.ts file and add them to controllers and providers array respectively.

src/students/students.module.ts

```
import { Module } from '@nestjs/common';
import { StudentsController } from './students.controller';
import { StudentsService } from './students.service';

@Module({
    controllers:[StudentsController],
    providers: [StudentsService]
})
export class StudentsModule {}
```

Shared modules

In Nest, modules are singletons by default, and thus you can share the same instance of any provider between multiple modules effortlessly.



Every module is automatically a shared module. Once created it can be reused by any module. Let's imagine that we want to share an instance of the StudentsService between several other modules. In order to do that, we first need to export the StudentsService provider by adding it to the module's exports array, as shown below:

src/students/students.module.ts

```
import { Module } from '@nestjs/common';
import { StudentsController } from './students.controller';
import { StudentsService } from './students.service';

@Module({
    controllers:[StudentsController],
    providers: [StudentsService],
    exports:[StudentsService]
})
export class StudentsModule {}
```

Now any module that imports the StudentsModule has access to the StudentsService and will share the same instance with all other modules that import it as well.

If we were to directly register the StudentsService in every module that requires it, it would indeed work, but it would result in each module getting its own separate instance of the StudentsService. This can lead to increased memory usage since multiple instances of the same service are created, and it could also cause unexpected behaviour, such as state inconsistency if the service maintains any internal state.

By encapsulating the StudentsService inside a module, such as the StudentsModule, and exporting it, we ensure that the same instance of StudentsService is reused across all modules that import StudentsModule. This not only reduces memory consumption but also leads to more predictable behaviour, as all modules share the same instance, making it easier to manage shared states or resources. This is one of the key benefits of modularity and dependency injection in frameworks like NestJS—allowing services to be efficiently shared throughout the application.

Global modules

If you have to import the same set of modules everywhere, it can get tedious. Unlike in Nest, Angular providers are registered in the global scope. Once defined, they're available everywhere. Nest, however, encapsulates providers inside the module scope. You aren't able to use a module's providers elsewhere without first importing the encapsulating module.

When you want to provide a set of providers which should be available everywhere out-of-the-box (e.g., helpers, database connections, etc.), make the module global with the @Global() decorator.

src/students/students.module.ts

```
import { Global, Module } from '@nestjs/common';
import { StudentsController } from './students.controller';
import { StudentsService } from './students.service';

@Global()
@Module({
    controllers:[StudentsController],
    providers: [StudentsService]
})
export class StudentsModule {}
```

The @Global() decorator makes the module global-scoped. Global modules should be registered only once, generally by the root or core module. In the above example, the StudentsService provider will be ubiquitous, and modules that wish to inject the service will not need to import the StudentsModule in their imports array.

Making everything global is not recommended as a design practice. While global modules can help reduce boilerplate, it's generally better to use the imports array to make a module's API available to other modules in a controlled and clear way. This approach provides better structure and maintainability, ensuring that only the necessary parts of the module are shared with others while avoiding unnecessary coupling between unrelated parts of the application.

Dynamic modules

Dynamic modules in Nest allow you to create modules that can be configured at runtime. This is especially useful when you need to provide flexible, customizable modules where the providers can be created based on certain options or configurations.

We will discuss this in detail in later chapters.

DRAFT

CRUD Generator

Throughout the life span of a project, when we build new features, we often need to add new resources to our application. These resources typically require multiple, repetitive operations that we have to repeat each time we define a new resource.

Let's imagine a real-world scenario, where we need to expose CRUD endpoints for 2 entities, let's say User and Product entities. Following the best practices, for each entity we would have to perform several operations, as follows:

- Generate a module (nest g mo) to keep code organized and establish clear boundaries (grouping related components)
- Generate a controller (nest g co) to define CRUD routes (or queries/mutations for GraphQL applications)
- Generate a service (nest g s) to implement & isolate business logic
- Generate an entity class/interface to represent the resource data shape
- Generate Data Transfer Objects (or inputs for GraphQL applications) to define how the data will be sent over the network

That's a lot of steps!

To help speed up this repetitive process, Nest CLI provides a generator (schematic) that automatically generates all the boilerplate code to help us avoid doing all of this, and make the developer experience much simpler.

Generating a new resource

In NestJS, a "**resource**" means a bunch of tools(such as controllers, services, and modules) that help handle common actions like creating, reading, updating, and deleting data for a specific thing in an application.

To create a new resource, simply run the following command in the root directory of your project:

```
nest g resource users
```

nest g resource command not only generates all the NestJS building blocks (module, service, controller classes) but also an entity class, DTO classes as well as the testing (.spec) files.

When executing above command it will prompt some questions like,

- What transport layer do you use?, there are many options like REST API, GraphQL etc..
- Would you like to generate CRUD entry points? Y/n

Here is the files generated when we specify **REST** API and **Yes** to generate CRUD entry point

- CREATE src/users/users.controller.ts
- CREATE src/users/users.controller.spec.ts
- CREATE src/users/users.module.ts
- CREATE src/users/users.service.ts
- CREATE src/users/users.service.spec.ts
- CREATE src/users/dto/create-user.dto.ts
- CREATE src/users/dto/update-user.dto.ts
- CREATE src/users/entities/user.entity.ts
- UPDATE src/app.module.ts

We can see below, that not only were all boilerplate mutations and queries created, but everything is all tied together. We're utilizing the UsersService, User Entity, and our DTO's.

Let's discuss some of important files in detail,

1. Controller:

   This manages how your app responds to requests. It's like the traffic cop deciding where each request should go.

2. Service:

   The service is where the smart work happens. It deals with the behind-the-scenes logic, like talking to a database or checking if the data is okay.

3. Module:

   Think of the module as a folder that keeps all these tools together for a specific job in your app.

4. DTO(Data Transfer Object):

   This is like a form you fill out to make sure the right information gets from one part of your app to another during these common actions.

5. Entity:

   If your app uses a database, an entity is like the blueprint for the data you're working with. Not every resource needs it, but sometimes it helps.

Creating RestAPI using NestJS

We are going to create a very basic RestAPI to perform crud operation on MySQL database using TypeORM and NestJS, here are the steps for the same.

Step 01: Create a new project with NestCLI using following command,

DRAFT

```
nest new crud-mysql-basic
```

Step 02: Install required dependencies like mysql2, typeorm, @nestjs/typeorm and @nestjs/config

```
npm install @nestjs/typeorm typeorm mysql2 @nestjs/config
```

Step 03: create a .env file and add following details in it,

.env

```
DB_HOST = localhost
DB_PORT = 3306
DB_USERNAME = root
DB_PASSWORD =
DB_DATABASE = nest_db_test
```

Step 04: in app.module.ts add ConfigModule and TypeOrmModule in imports array

app.module.ts

```
...

@Module({
  imports: [
    ConfigModule.forRoot(),
    TypeOrmModule.forRootAsync({
      imports: [ConfigModule],
      useFactory: (configService : ConfigService)=>({
        type: 'mysql',
        host: configService.get('DB_HOST'),
        port: configService.get('DB_PORT'),
        username: configService.get('DB_USERNAME'),
        password: configService.get('DB_PASSWORD'),
        database: configService.get('DB_DATABASE'),
        entities: [],
        synchronize: true
      }),
      inject: [ConfigService]
    })
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

Step 05: create a resource with NestCLI using following command,

```
nest generate resource student
```

This will prompt:

```
? What transport layer do you use? (Use arrow keys)
> REST API
  GraphQL (code first)
  GraphQL (schema first)
  Microservice (non-HTTP)
  WebSockets
```

```
? Would you like to generate CRUD entry points? (Y/n) Y
```

This will create 8 files and update app.module.ts

```
CREATE src/student/student.controller.ts (980 bytes)
CREATE src/student/student.controller.spec.ts (606 bytes)
CREATE src/student/student.module.ts (271 bytes)
CREATE src/student/student.service.ts (675 bytes)
CREATE src/student/student.service.spec.ts (485 bytes)
CREATE src/student/dto/create-student.dto.ts (34 bytes)
CREATE src/student/dto/update-student.dto.ts (185 bytes)
CREATE src/student/entities/student.entity.ts (25 bytes)
UPDATE src/app.module.ts (1036 bytes)
```

Step 06: Modify src/student/entities/student.entity.ts file

Student.entity.ts

```
import { Entity, Column, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class Student {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  StudentName: string;

  @Column()
  StudentEmail: string;

  @Column()
  StudentSPI: number
}
```

Step 07: Modify src/student/dto/create-student.dto.ts

create-student.dto.ts

```
export class CreateStudentDto {
  StudentName: string;
  StudentEmail: string;
  StudentSPI: number
}
```

Step 08 Connect entity in src/student/student.module.ts

student.module.ts

```
… … …
@Module({
  … … …
  imports:[TypeOrmModule.forFeature([Student])],
  … … …
})
… … …
```

Step 09: Add StudentModule to imports array in app.module.ts

Note:This step will be done automatically, you just need to check.

Step 10: Inject Student Repository in src/student/student.service.ts

student.module.ts

```
… … …
export class StudentService {
  … … …
  constructor(
    @InjectRepository(Student)
      private studentRepo: Repository<Student>,
    ) {}

  … … …
}
… … …
```

Step 11: Update methods to perform database operations in src/student/student.service.ts

student.service.ts

```
import { Injectable } from '@nestjs/common';
import { CreateStudentDto } from './dto/create-student.dto';
import { UpdateStudentDto } from './dto/update-student.dto';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Student } from './entities/student.entity';

@Injectable()
export class StudentService {

  constructor(
      @InjectRepository(Student)
      private studentRepo: Repository<Student>,
    ) {}

  create(createStudentDto: CreateStudentDto) {
    return this.studentRepo.save(createStudentDto);
  }

  findAll() {
    return this.studentRepo.find();
  }

  findOne(id: number) {
    return this.studentRepo.findOneBy({id});
  }

  update(id: number, updateStudentDto: UpdateStudentDto) {
    return this.studentRepo.update(id, updateStudentDto);
  }

  remove(id: number) {
    return this.studentRepo.delete(id);
  }
}
```